Week 20 | Sensor-to-field learning with missing observations

Formulate a comparison before fitting

The scientific question is whether a set-based attention model is useful when the available sensor set changes. A comparison on one fixed, noiseless sensor vector mainly tests function fitting on that particular vector. It does not exercise the cardinality flexibility often used to motivate attention.

We therefore compare complete observations with a predetermined half-sensor dropout condition. The protocol fixes the training cases, validation case, retained evaluation case, sixteen QR/D-optimal sensors and observation noise. The evaluation condition is declared before looking at its results. These are already-inspected teaching cases; the report does not label Re105 a newly blind test.

Four estimators, one target representation

All learned reconstructions use the same training-fitted rank-eight POD decoder. The Sensor-set model embeds normalized value-coordinate triples, applies self-attention and uses learned invariant pooling to produce modal coefficients. The release POD-DeepONet uses a fixed sixteen-input branch network with a 208-unit hidden layer and the same fixed POD trunk. Its 5208 parameters approximately match the attention model's 5192. Ridge maps the same sensor vector linearly to coefficients. Gappy POD solves the observed basis directly.

$$\hat{u} = \bar{u} + \Phi \,\mathrm{diag}(s) f_\theta(y, x)$$

This classroom POD-DeepONet is not a reproduction of every published DeepONet variant. Its shared fixed decoder makes representation capacity explicit. Neither model predicts pressure, enforces a no-slip boundary nor learns a general full-domain Navier-Stokes solution. The wake ROI excludes the cylinder.

Estimator	Handles available coordinates directly?	Missing-observation policy
Sensor-set attention	Yes	Remove value-coordinate tokens together
Fixed POD-DeepONet branch	No	Impute missing normalized values by zero
Ridge branch	No	Same training-mean imputation
Gappy POD	Yes	Solve only the available basis rows

What the dropout experiment can establish

The set model is trained with variable-cardinality augmentation. Fixed-vector controls retain their original interfaces and receive declared imputation at evaluation. This is a comparison of systems with different adaptation mechanisms, not a capacity-matched architecture-only ablation. Parameter count, training time and the augmentation rule are part of the record.

A classical method remains a strong comparator because gappy POD naturally accepts a subset of available sensors. Missing observations do not imply that attention should win. A network may be permutation invariant yet inaccurate, or robust to dropout yet worse than a linear estimator at every cardinality. Those outcomes should remain in the table.

The current release tests complete and fixed half-sensor conditions. Arbitrarily relocated sensors and irregular geometries are separate extensions, not implied successes. To study relocation, train with coordinate variation and freeze a new-location evaluation protocol; do not relabel the existing dropout experiment.

Optimization is an experimental variable

The old 180-step comparison selected all neural checkpoints at its final evaluation point. The revised protocol allows a larger ceiling, records full training and validation traces, and uses validation patience. Each record includes selected step, executed steps, maximum allowed steps, parameter count and wall-clock fit seconds.

A selected checkpoint at the ceiling is a budget warning. It does not logically prove undertraining: the final point might be the true optimum of the observed interval. Nor does selecting an earlier point prove convergence. Inspect curve shape, then perform a predeclared budget sensitivity study using validation for all choices. Never extend only the model whose evaluation score looked disappointing.

$$\theta^{*} = \arg\min_{\theta_k \in \mathrm{C}} \mathrm{L}_{validation}(\theta_k)$$

The set C consists of recorded candidate checkpoints. Evaluation fields are absent from the fitting interface. A poisoned-evaluation test should leave training weights and selected checkpoint unchanged. This is stronger evidence than checking that a constant dictionary contains the intended case numbers.

Interpret field error relative to the decoder floor

For an orthonormal basis and a prediction inside its affine span, the error decomposes into orthogonal representation and in-subspace components. Their squared norms add; their unsquared norms generally do not. Use the same truth norm as denominator for all three quantities.

$$\|\hat{u} - u\|^2 = \|u_r - u\|^2 + \|\hat{u} - u_r\|^2$$

Suppose the representation floor is 4% and the learned in-subspace error is 3%. Total field error is 5%, not 7%. Reducing the learned component to 1% changes the total only to about 4.12%. A small change in field error can therefore conceal a large improvement in coefficient prediction, while a near-floor result leaves little room for further benefit without changing representation.

The equality relies on orthogonality in the metric being used. Here the spatial grid has uniform cell area, so unweighted Euclidean norms have the same relative weighting as area-weighted L2. On a nonuniform mesh, use the corresponding weighted orthonormal basis and inner product.

Worked implementation and student responsibilities

The notebook trains a short 120-step model as an interactive exercise and labels that budget explicitly. It then loads the full retained comparison, reconstructs a stored predicted field and independently recomputes its relative error. The short classroom run must not be mistaken for the release evidence.

Four tasks implement consistent token dropout, count trainable parameters, diagnose a selected-at-ceiling record and train a model with validation-only checkpoint selection under a fixed update ceiling. The retained branch has widths 16-208-8, including biases; calculate its trainable capacity from the instantiated model rather than relying on an outdated small example. The selection task intentionally contains no evaluation score.

The separate three-seed paired SensorSet ablation holds architecture, initialization, observations, POD basis, optimizer, validation objective and budget ceiling fixed. With all sensors, augmented/plain errors are 13.55/6.08%, 14.43/4.65% and 11.61/5.73%; with the fixed half-sensor subset they are 27.16/60.46%, 24.99/39.41% and 22.62/36.89%. Augmentation improves missing-sensor error in these three pairs while sacrificing full-sensor accuracy. The selection objective uses all-sensor validation; actual early-stopped budgets differ. This isolates the training recipe within SensorSet and does not rank architectures or establish statistical significance. Full traces and paired evidence are in results/transformer_sensor_ablation.

Reporting and reading

Report every seed and condition; show validation curves; distinguish model size, update count and measured time. Add the decoder floor and a separate error color scale to field figures. State whether dropout occurred during training. A conclusion should name the exact condition and comparator rather than saying that attention is universally better or worse.

Course background: the Week 4/5 POD-DeepONet and modal-sensing laboratories. Architecture context: Lu et al., Learning nonlinear operators via DeepONet, https://doi.org/10.1038/s42256-021-00302-5; Wu et al., Transolver (2024), https://arxiv.org/abs/2402.02366. The latter is a research comparison for discussion, not an architecture claimed reproduced by this compact POD-decoder experiment.

Printable laboratory worksheet

Task 1: Consistent sensor-token dropout

Remove every other sensor from values and coordinates together; never reinterpret a missing value as an observed zero.

Interface contract: Input tokens:[batch,sensors,features] is a NumPy array. Return tokens[:,::2,:], preserving all features for every retained sensor.

Task 2: Capacity accounting

Count trainable parameters; compare the attention model with the retained 16-208-8 branch network (Protocol.branch_width=208).

Interface contract: Input is a torch.nn.Module. Return a Python integer counting only parameters with requires_grad=True, including biases.

Task 3: Training budget diagnosis

Flag a record whose selected checkpoint reaches its ceiling. This diagnoses an unresolved budget limit, not proof of convergence or its absence.

Interface contract: record is a dictionary with integer selected_step and ceiling. Return True exactly when selected_step==ceiling; a near-ceiling checkpoint is False.

Task 4: Implement validation-only checkpoint selection

Train a supplied small model using full-batch SGD and mean squared error. Evaluate validation at step zero and after each update; retain and restore the best state. Return selected step, executed steps and validation trace. Neither evaluation data nor evaluation loss is an input. Compare two learning rates and explain any step-zero selection.

Interface contract: steps is a nonnegative integer, lr a positive scalar. Inputs are dimension-compatible Torch tensors. Use full-batch SGD with mean MSE. Evaluate step zero and every update; select the earliest minimum validation loss; restore it and leave model in evaluation mode. Return dictionary selected_step (int), executed_steps (steps), validation_mse (float minimum), trace (list of dictionaries with step and validation_mse for 0..steps). Negative steps or nonpositive lr must raise ValueError.

Before submitting

Restart the kernel and run the worked examples and completed tasks. Confirm that all four coding checks pass and that no retained data or results file changed. Include the requested interpretation, identify every quantity fitted from training data, and separate test observations from decisions made using validation. Record unresolved budget or representation limits rather than removing inconvenient runs.

Assessment and worked interpretation

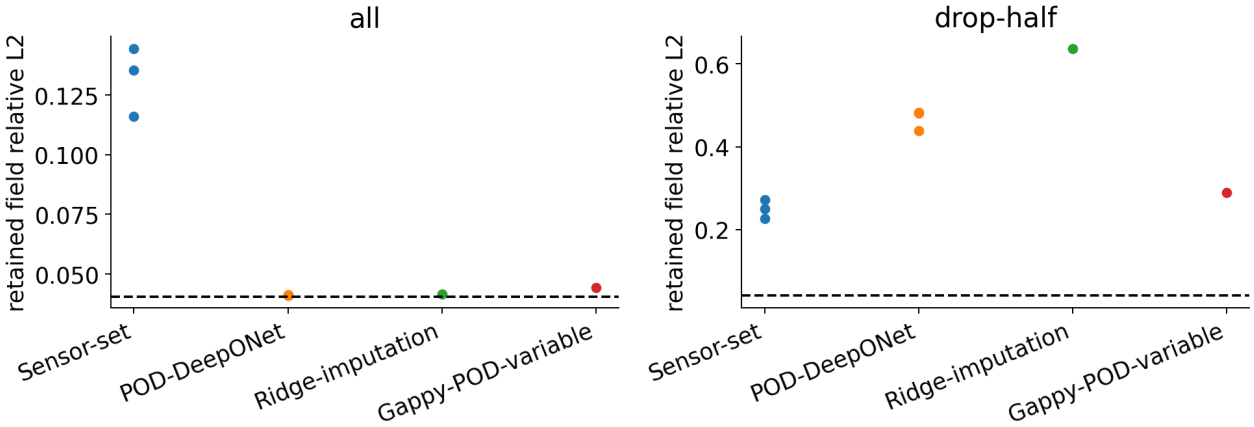
Coding: Tasks 1–3 carry 5 points each; the integrated Task 4 carries 25. Derivation (25): derive permutation invariance of pooled sensor attention and distinguish cardinality dropout from mean imputation. Interpretation (25): give evidence and limitations. Reproducibility (10): identify inputs, split, checkpoint, environment and every failed seed. Total 100. Passing function checks earns only the coding allocation.

Worked full-credit reasoning: An attention set is equivariant before pooling and invariant afterward. Dropping tokens reduces cardinality, whereas replacing their values with zero retains a different input. Compare actual parameter counts, selection steps and validation criteria before attributing accuracy to architecture.

Misconception to reject: “A model that reaches its update ceiling is necessarily converged, and equal update caps guarantee equal compute.” Explain the failure in one equation or counterexample. An observation without protocol and limitation earns at most half of interpretation credit. Instructor judgment and a timed student pilot remain necessary.

Timing estimate: a 120-minute guided lab and roughly 3–5 hours including preparation and written analysis. This is an editorial estimate; a timed student pilot remains open.

Executed evidence and interpretation



This figure is generated from the retained protocol. Use the notebook to inspect all seeds and recompute the plotted quantities. The earlier lecture sections specify what the diagnostic can and cannot establish.